2xxx Successful HTTP Status Codes (200, 201, 202, 204)

What are 2xx Status Codes?

Meaning: The client's request was successfully received, understood, and accepted by the server.

Category: 2xx (Successful)

These status codes confirm that the action the client intended to perform was completed successfully.

2xxx Status Codes

<u>Status Code 200:</u>

- **Name:** OK
- **Meaning:** Request succeeded. Response includes requested data
- **When to Use:** GET (fetching data), PUT (updating), successful POST
- **ASP.NET Core Helper:** Ok(data)

<u>Status Code 201:</u>

- **Name:** Created
- **Meaning:** Request succeeded and a new resource was created
- **When to Use:** POST (creating new record). Should include Location header
- **ASP.NET Core Helper:** CreatedAtAction()

<u>Status Code 202:</u>

- **Name:** Accepted
- **Meaning:** Request accepted but processing may not be complete immediately
- **When to Use:** Asynchronous operations (background jobs, batch processing)
- **ASP.NET Core Helper:** Accepted()

Status Code 204:

- **Name:** No Content
- **Meaning:** Request succeeded but no content to return (response body empty)
- **When to Use:** DELETE operation, or PUT with no response body
- **ASP.NET Core Helper:** NoContent()

Why These Status Codes Matter

Helper Method	Returns	Status Code
Ok(data)	Data + 200 OK	200
CreatedAtAction()	Data + Location header + 201 Created	201
Accepted()	Message + 202 Accepted	202
NoContent()	Empty body + 204 No Content	204

Now we are going to set up a project ‘ClothsApi1_SuccessStatusCodes’ to represent the above concepts,

Project Setup

Create the project:

Command: dotnet new webapi -n ClothsApi1_SuccessStatusCodes -f net8.0

Navigate to the project:

Command: cd ClothsApi1_SuccessStatusCodes

Install Packages

Before installing the packages, we should make sure that we are in the project folder ‘ClothsApi1_SuccessStatusCodes’ in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

ClothsApi1_SuccessStatusCodes.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0"
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Cloth Model

Path: ClothsApi1_SuccessStatusCodes/Models/Cloth.cs

Cloth.cs

Models > Cloth.cs > ...

```
namespace ClothsApi1_SuccessStatusCodes.Models
{
    0 references
    public class Cloth
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Name { get; set; } = string.Empty;
        0 references
        public string Category { get; set; } = string.Empty;
        0 references
        public string Size { get; set; } = string.Empty;
        0 references
        public string Color { get; set; } = string.Empty;
        0 references
        public decimal Price { get; set; }
        0 references
        public bool InStock { get; set; }
    }
}
```

Step 2: Create Cloth Service (Async)

Path: ClothsApi1_SuccessStatusCodes/Services/ClothService.cs

ClothService.cs

Services > ClothService.cs > ...

```
using ClothsApi1_SuccessStatusCodes.Models;

namespace ClothsApi1_SuccessStatusCodes.Services
{
    0 references
    public static class ClothService
    {
        // In-memory data (7 cloth items)
        6 references
        private static readonly List<Cloth> _cloths = new()
        {
            new Cloth {
                Id = 1,
                Name = "Cotton T-Shirt",
                Category = "Men",
                Size = "M",
                Color = "Blue",
                Price = 599,
                InStock = true
            },
            new Cloth {
                Id = 2,
                Name = "Denim Jacket",
                Category = "Women",
                Size = "L",
                Color = "Black",
                Price = 2499,
                InStock = true
            },
```

```
            new Cloth {
                Id = 3,
                Name = "Formal Shirt",
                Category = "Men",
                Size = "XL",
                Color = "White",
                Price = 1299,
                InStock = false
            },
```

```
            new Cloth {
                Id = 4,
                Name = "Sports Shoes",
                Category = "Men",
                Size = "9",
                Color = "Grey",
                Price = 3499,
                InStock = true
            },
            new Cloth {
                Id = 5,
                Name = "Silk Saree",
                Category = "Women",
                Size = "Free",
                Color = "Red",
                Price = 4999,
                InStock = true
            },
```

```
            new Cloth {
                Id = 6,
                Name = "Kids Hoodie",
                Category = "Kids",
                Size = "S",
                Color = "Yellow",
                Price = 899,
                InStock = true
            },
            new Cloth {
                Id = 7,
                Name = "Leather Belt",
                Category = "Accessories",
                Size = "32",
                Color = "Brown",
                Price = 399,
                InStock = false
            }
        };
```

```

        // 200 OK: Returns all cloths
        0 references
        public static async Task<List<Cloth>> GetAllClothsAsync()
        {
            await Task.Delay(100);
            return _cloths;
        }
```

```
        // 200 OK: Returns single cloth by ID
        0 references
        public static async Task<Cloth?> GetClothByIdAsync(int id)
        {
            await Task.Delay(50);
            return _cloths.FirstOrDefault(c => c.Id == id);
        }

        // 201 Created: Adds new cloth
        0 references
        public static async Task<Cloth> AddClothAsync(Cloth newCloth)
        {
            await Task.Delay(100);
            newCloth.Id = _cloths.Max(c => c.Id) + 1;
            _cloths.Add(newCloth);
            return newCloth;
        }
```

```

        // 204 No Content: Deletes cloth
        0 references
        public static async Task<bool> DeleteClothAsync(int id)
        {
            await Task.Delay(80);
            var cloth = _cloths.FirstOrDefault(c => c.Id == id);
            if (cloth == null) return false;
            return _cloths.Remove(cloth);
        }

        // 202 Accepted: Simulates async background processing
        0 references
        public static async Task<bool> ProcessBulkOrderAsync()
        {
            await Task.Delay(500);
            return true;
        }
    }
}

```

Step 3: Program.cs (Minimal)

Path: ClothsApi1_SuccessStatusCodes/Program.cs

Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Step 4: Controller (All 2xx Status Codes)

Path: ClothsApi1_SuccessStatusCodes/Controllers/ClothController.cs

ClothController.cs

Controllers > ClothController.cs > ...

```
using ClothsApi1_SuccessStatusCodes.Models;
using ClothsApi1_SuccessStatusCodes.Services;
using Microsoft.AspNetCore.Mvc;

namespace ClothsApi1_SuccessStatusCodes.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class ClothController : ControllerBase
    {
        // ============================================================
        // 200 OK - GET /api/cloth/5
        // Returns cloth data with 200 OK status
        // Use Ok() helper method
        // ============================================================
        [HttpGet("{id}")]
        1 reference
        public async Task<ActionResult<Cloth>> GetClothById(int id)
        {
            var cloth = await ClothService.GetClothByIdAsync(id);

            if(cloth == null)
                return NotFound($"Cloth with {id} not found");


            // Returns 200 OK with cloth data in response body
            return Ok(cloth);
        }

```

```
        // ============================================================
        // 200 OK - GET /api/cloth
        // Returns all cloths with 200 status
        // ============================================================
        [HttpGet]
        0 references
        public async Task<ActionResult<List<Cloth>>> GetAllCloths()
        {
            var cloths = await ClothService.GetAllClothsAsync();

            // Returns 200 OK with list of cloths
            return Ok(cloths);
        }

```

```
42        // ============================================================
43        // 201 Created - POST /api/cloth
44        // Creates new cloth, returns 201 with Location header
45        // Use CreatedAtAction() helper method
46        // ============================================================
47        [HttpPost]
          0 references
48        public async Task<ActionResult<Cloth>> CreateCloth
49        ([FromBody] Cloth newCloth)
50        {
51            if (string.IsNullOrEmpty(newCloth.Name))
52                return BadRequest("Cloth name is required");
53
54            var createdCloth = await ClothService.AddClothAsync(newCloth);
55
56            // Returns 201 Created with:
57            // - Location header pointing to GetClothById endpoint
58            // - Created cloth data in response body
59            return CreatedAtAction(
60                actionName: nameof(GetClothById),
61                routeValues: new { id = createdCloth.Id},
62                value: createdCloth
63            );
64        }
65
```

```
66        // ============================================================
67        // 202 Accepted - POST /api/cloth/bulk-order
68        // Accepts async operation, returns 202 with status message
69        // Use Accepted() helper method
70        // ============================================================
71        [HttpPost("bulk-order")]
          0 references
72        public async Task<ActionResult> ProcessBulkOrder()
73        {
74            // Simulate async background processing
75            await ClothService.ProcessBulkOrderAsync();
76
77            // Returns 202 Accepted with status message
78            return Accepted(new
79            {
80                Message = "Bulk order accepted. Processing in background.",
81                EstimatedCompletion = "5 minutes",
82                TrackingId = Guid.NewGuid().ToString()
83            });
84        }
85
```

```
        // ==============================================================
        // 204 No Content - DELETE /api/cloth/5
        // Delets cloth, returns empty body with 204
        // Use NoContent() helper method
        // ==============================================================
        [HttpDelete("{id}")]
        0 references
        public async Task<ActionResult> DeleteCloth(int id)
        {
            var deleted = await ClothService.DeleteClothAsync(id);

            if (!deleted)
                return NotFound($"Cloth with ID {id} not found");

            // Returns 204 No Content (empty response body)
            return NoContent();
        }
    }
}
```

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (ClothsApi1_SuccessStatusCodes) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi1_SuccessStatusCodes> dotnet
build
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi1_SuccessStatusCodes -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\Cloth
  sApi1_SuccessStatusCodes\bin\Debug\net8.0\ClothsApi1_SuccessStatusCodes.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.28
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi1_SuccessStatusCodes> dotnet
watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/ho
t-reload.
  💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi1_SuccessStatusCodes -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\Cloth
  sApi1_SuccessStatusCodes\bin\Debug\net8.0\ClothsApi1_SuccessStatusCodes.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5242
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi1_Succe
ssStatusCodes
```

ClothsApi1_SuccessStatusCodes 1.0 OAS 3.0

http://localhost:5242/swagger/v1/swagger.json

Cloth

GET /api/Cloth/{id}

DELETE /api/Cloth/{id}

GET /api/Cloth

POST /api/Cloth

POST /api/Cloth/bulk-order

Schemas

Cloth >

Test 1: 200 OK – Get Single Cloth

Request: GET http://localhost:5242/api/Cloth/3

Response (200 OK):

Request URL

http://localhost:5242/api/Cloth/3

Server response

Code

200

Details

Response body


```
{
  "id": 3,
  "name": "Formal Shirt",
  "category": "Men",
  "size": "XL",
  "color": "White",
  "price": 1299,
  "inStock": false
}
```


Download

Test 2: 200 OK – Get Single Cloth

Request: GET http://localhost:5242/api/Cloth

Response (200 OK):

Request URL

http://localhost:5242/api/Cloth

Server response

Code

200

Details

Response body


```
[
  {
    "id": 1,
```


localhost:5242/api/Cloth

Import favorites | Active nowCSE Ru... | YouTube | Other favorit

Pretty-print


```
[{"id":1,"name":"Cotton T-Shirt","category":"Men","size":"M","color":"Blue","price":599,"inStock":true},
{"id":2,"name":"Denim Jacket","category":"Women","size":"L","color":"Black","price":2499,"inStock":true},
{"id":3,"name":"Formal Shirt","category":"Men","size":"XL","color":"White","price":1299,"inStock":false},
{"id":4,"name":"Sports Shoes","category":"Men","size":"9","color":"Grey","price":3499,"inStock":true},
{"id":5,"name":"Silk Saree","category":"Women","size":"Free","color":"Red","price":4999,"inStock":true},
{"id":6,"name":"Kids Hoodie","category":"Kids","size":"S","color":"Yellow","price":899,"inStock":true},
{"id":7,"name":"Leather Belt","category":"Accessories","size":"32","color":"Brown","price":399,"inStock":false}]
```

Test 3: 201 Created – Create New Cloth

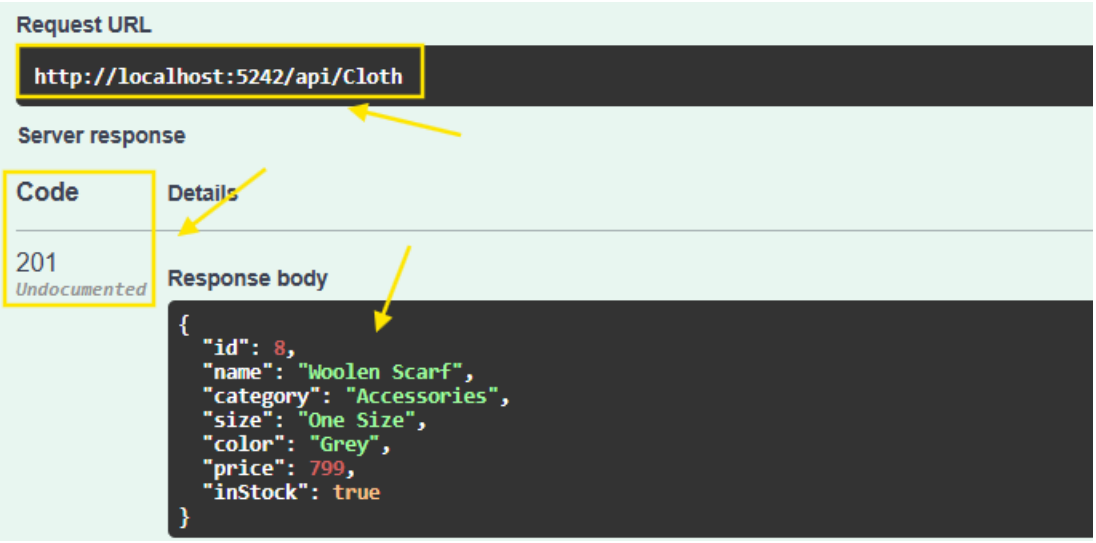
Request: **POST http://localhost:5242/api/Cloth**

Content-Type: application/json

Content-Body:


```
{
    "name": "Woolen Scarf",
    "category": "Accessories",
    "size": "One Size",
    "color": "Grey",
    "price": 799,
    "inStock": true
}
```


Response (201 Created):



Test 4: 202 Accepted – Async Bulk Order

Request: GET http://localhost:5242/api/Cloth/bulk-order

Response (202 Accepted):

Curl

```
curl -X 'POST' \
  'http://localhost:5242/api/Cloth/bulk-order' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
http://localhost:5242/api/Cloth/bulk-order
```

Server response

Code	Details
202 *Undocumented*	Response body

```
{
  "message": "Bulk order accepted. Processing in background.",
  "estimatedCompletion": "5 minutes",
  "trackingId": "4b47f62b-fa40-4750-b74f-791ab8378ee3"
}
```

Test 5: 204 No Content – Delete Cloth

Request: DELETE http://localhost:5242/api/Cloth/7

Response (204 No Content):

Curl

```
curl -X 'DELETE' \
  'http://localhost:5242/api/Cloth/7' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5242/api/Cloth/7
```

Server response

Code	Details
204 *Undocumented*	Response headers

```
date: Sat,25 Apr 2026 19:13:18 GMT
server: Kestrel
```

Test 6: 200 OK – Verify Deletion (Cloth 7 gone)

Request: **GET http://localhost:5242/api/Cloth/7**

Response (404 Not Found):

Curl

```
curl -X 'GET' \
  'http://localhost:5242/api/Cloth/7' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5242/api/Cloth/7
```

Server response

Code	Details
404 *Undocumented*	Error: Not Found

Response body

```
Cloth with 7 not found
```

Kay Takeaways

Status Codes	Helper Method	When to Use	URL Pattern
200	Ok(data)	Successful GET, PUT, POST with data	/api/cloth, /api/cloth/5
201	CreatedAtAction()	POST that creates new resource	/api/cloth
202	Accepted()	Async operations (background jobs)	/api/cloth/bulk-order
204	NoContent()	DELETE or PUT with no response body	/api/cloth/5